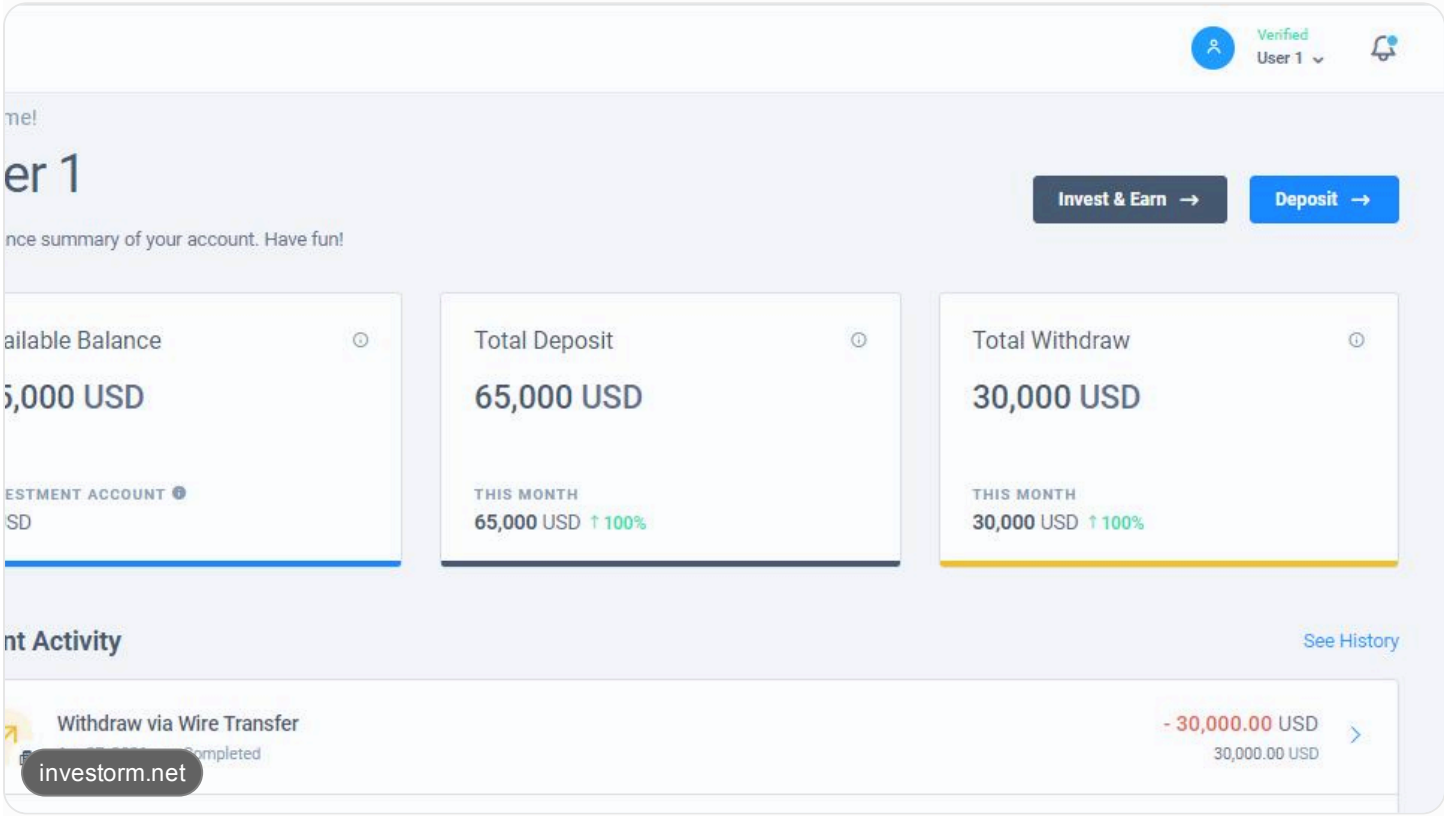
TypeScript Hands-on Training: Banking Application Project

For your 9-hour training session with second-year UG Computer Science students, I recommend developing a **Banking Transaction Management System using TypeScript**. Students will build the application step by step, learning each TypeScript concept while implementing a real application feature.

The project will be suitable for classroom demonstrations, guided coding, practical exercises, and a final student assessment.

1. Project overview



Customer Management

Create and manage bank customer profiles.

Account Management

Create accounts and display account balances.

Deposit & Withdrawal

Validate transactions and update balances.

Transaction History

Display deposits, withdrawals and transfers.

Application features

Students will build a working application with:

1. Customer registration and customer listing.

2. Bank account creation with an initial balance.
3. Account balance enquiry.
4. Deposit money.
5. Withdraw money with insufficient-balance validation.
6. Transfer money between two accounts.
7. Transaction history with transaction type, amount, date and status.
8. Search accounts by account number.
9. Input validation and error handling.
10. A simple browser-based user interface.

Training scope: This will be a front-end educational simulation using TypeScript, Vite and browser storage. It will not connect to an actual bank, process real money, or provide production banking security.

2. Mapping the seven TypeScript topics to the project

TypeScript topic	Banking application implementation
Introduction to TypeScript	Project setup, `.ts` files, compilation and browser execution
Type System	`string`, `number`, `boolean`, arrays, type inference and strict checking
Functions	Deposit, withdrawal, transfer, search and validation functions
Object-Oriented Programming	`Customer`, `BankAccount` and `Bank` classes
Interfaces and Types	Customer, account, transaction and service contracts
Advanced Types	Union types, literal types, optional properties, narrowing and generics
TypeScript with Build Tools	npm, TypeScript compiler, Vite, development server and production build

3. Download the complete project source code

TypeScript Banking Application — Source Code ZIP

TypeScript · Vite · HTML · CSS · Browser storage

Complete project, README, setup instructions and teaching plan

Download the complete project ZIP ↗

Copy execution commands

Project folder structure

The ZIP contains the following source files.

typescript_banking_training/

src/types.ts

Interfaces, type aliases, union types and generic result type.

src/bank.ts

Bank class, customer registration, account creation, deposits, withdrawals and transfers.

src/storage.ts

Repository interface and localStorage implementation.

src/main.ts

Browser interface, form handling, account search and transaction history.

src/style.css

Responsive dashboard styling, forms, tables and notifications.

package.json, tsconfig.json, index.html

Dependencies, compiler configuration and application entry point.

README.md

Installation, 9-hour schedule, teaching notes, testing checklist and assessment rubric.

4. Nine-hour trainer schedule

I suggest structuring the training as **six hours of concept-driven development, one hour of build tools, and two hours of integration and assessment**. The timings below total exactly nine hours.

45 min

Session 1

Introduction to TypeScript

- What TypeScript is and why it is useful.
- TypeScript vs JavaScript.
- Install Node.js, npm and project dependencies.
- Introduce the project structure and Vite.
- Run the starter application.

Practical output: running banking dashboard.

60 min

Session 2

Type System

- Primitive types, arrays and type inference.
- Create `Customer`, `BankAccount` and `Transaction` models.
- Use optional properties and strict compiler checking.

Practical output: strongly typed banking data.

60 min

Session 3

Functions

- Parameters, return types and optional parameters.
- Implement amount validation.
- Develop deposit and withdrawal logic.
- Handle invalid amounts and insufficient balance.

Practical output: working deposit and withdrawal operations.

---

75 min

Session 4

Object-Oriented Programming

- Classes, objects and constructors.
- Public and private members.
- Encapsulation of banking data.
- Implement the `Bank` class and its methods.

Practical output: centralised banking business logic.

---

60 min

Session 5

Interfaces and Types

- Define interfaces for customers, accounts and transactions.
- Create a repository interface for persistence.
- Use type aliases for account status and transaction types.

Practical output: consistent data structures across the application.

---

75 min

Session 6

Advanced Types

- Union and literal types.
- Discriminated unions for transaction requests.
- Generic `OperationResult<T>`.
- Type narrowing and safe error handling.
- Implement transfers between accounts.

Practical output: validated transfers and typed transaction results.

---

60 min

Session 7

TypeScript with Build Tools

- `package.json` and npm scripts.
- `tsconfig.json` and strict mode.
- Vite development server.
- Type-checking and production build.

Practical output: successful type-check and production build.

---

75 min

Guided Integration Lab

Session 8	• Connect customer registration and account creation. • Test deposits, withdrawals and transfers. • Display transaction history. • Verify balances and persistence after refresh. Practical output: complete banking application workflow.
30 min Session 9	**Testing and Assessment** • Test invalid and valid transactions. • Check TypeScript compiler errors. • Ask students to explain classes, interfaces and unions. • Give a short independent coding challenge. Practical output: tested project and student demonstration.

5. Step-by-step project execution

Step 1 — Install the prerequisites

Install Node.js and VS Code. Extract the ZIP and open the project folder in VS Code.

Open the VS Code terminal using **Terminal** → **New Terminal**.

Step 2 — Install dependencies

```bash
npm install
```

This installs TypeScript and Vite as development dependencies.

Step 3 — Start the application

```bash
npm run dev
```

Open the local URL shown in the terminal, usually `http://localhost:5173`.

The application starts with two sample customers and accounts, allowing students to begin practising transactions immediately.

Step 4 — Explore the banking workflow

Register a customer

Enter a name, valid email and phone number.

↓

Open an account

Select a customer and enter an opening balance.

↓

Perform transactions

Deposit, withdraw or transfer funds with validation.

↓

Review the transaction ledger

Verify amounts, account balances, descriptions and timestamps.

Step 5 — Check TypeScript errors

```bash
npm run typecheck
```

Explain to students that type-checking identifies errors before the application is built. For example, attempting to assign a string to a numeric balance should produce a TypeScript error.

Step 6 — Build the production application

```bash
npm run build
```

Vite creates the production files in the `dist` directory. You can preview the build with:

```bash
npm run preview
```

6. Important TypeScript examples to teach from the project

These examples can be demonstrated directly from the supplied source code.

Example A: Type system

TypeScript

```typescript
interface BankAccount {
  accountNumber: string;
  balance: number;
  status: "active" | "frozen" | "closed";
}

const account: BankAccount = {
  accountNumber: "10000001",
  balance: 5000,
  status: "active"
};
```

Explain how TypeScript checks each property and restricts the account status to three permitted values.

Example B: Functions

TypeScript

```typescript
function calculateNewBalance(
  currentBalance: number,
  amount: number,
  operation: "deposit" | "withdrawal"
): number {
  if (amount <= 0) {
    throw new Error("Amount must be positive.");
  }

  if (operation === "withdrawal" && amount > currentBalance) {
    throw new Error("Insufficient balance.");
  }

  return operation === "deposit"
    ? currentBalance + amount
    : currentBalance - amount;
}
```

Ask students to test deposits, withdrawals, zero amounts and insufficient balance. Explain that this function is a teaching example; the supplied `Bank` class contains the integrated transaction logic.

Example C: Object-oriented programming

TypeScript

```
class SimpleBankAccount {
  constructor(
    public readonly accountNumber: string,
    private balance: number
  ) {}

  deposit(amount: number): void {
    if (amount <= 0) {
      throw new Error("Amount must be positive.");
    }

    this.balance += amount;
  }

  getBalance(): number {
    return this.balance;
  }
}
```

Explain the constructor, object creation, `private` data and public methods. In the main project, the `Bank` class coordinates multiple accounts and their transactions.

Example D: Advanced types

TypeScript

```
type TransactionRequest =
  | { kind: "deposit"; amount: number }
  | { kind: "withdrawal"; amount: number };

function describeTransaction(request: TransactionRequest): string {
  switch (request.kind) {
    case "deposit":
      return `Deposit: ${request.amount}`;

    case "withdrawal":
      return `Withdrawal: ${request.amount}`;
```

```
  }
}
```

Explain how TypeScript narrows the type based on `request.kind`, allowing safer code without relying on unstructured strings.

7. Practical test cases for students

Test case	Expected result
Register a customer with valid details	Customer created
Register the same email again	Validation error
Deposit ₹500 into a valid account	Balance increases by ₹500
Withdraw ₹200 from a sufficient balance	Balance decreases by ₹200
Withdraw more than the available balance	Transaction rejected
Transfer ₹300 to another valid account	Source decreases and destination increases
Transfer to the same account	Transaction rejected
Submit a zero or negative amount	Transaction rejected
Refresh the browser	Demo data remains available
Run `npm run typecheck`	No TypeScript errors

8. Final student assignment

After the guided development, give students one independent enhancement to implement.

Choose a practical challenge

- (●) Add account freezing and prevent transactions on frozen accounts.
- ( ) Limit each withdrawal to ₹10,000.
- ( ) Add transaction filtering by transaction type.
- ( ) Generate a mini bank statement for a selected account.

Generate assignment and solution

Trainer's final recommendation

Use the supplied ZIP as your starter project, but **teach the code in dependency order**: types first, then functions, then the `Bank` class, then the user interface and persistence. This will help students understand why each TypeScript feature exists rather than merely copying a completed application.

One important limitation to explain in class: this application stores demo data in the browser's `localStorage`. It is suitable for a 9-hour TypeScript workshop, but it does not implement real banking security, server-side authorization, a database, or production-grade financial transaction processing.